

Important Code Reference: cloudflare/portfolio-ai-worker.js

A focused code/control-file explanation covering purpose, owner layer, APIs, variables, implementation signals, source excerpts, and debugging rules.

Item	Explanation
File	cloudflare/portfolio-ai-worker.js
Category	JavaScript source
Folder role	Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.
Size	10,435 bytes
Extension	.js
MIME type	application/javascript
Text lines	276

Why this file exists

- Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.

How it is used

Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/portfolio-ai	Line 228. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Network request at line 2	Frontend-backend communication or public web/API calls. Evidence: "https://mauriceotieno.com",
Network request at line 3	Frontend-backend communication or public web/API calls. Evidence: "https://www.mauriceotieno.com",
Network request at line 4	Frontend-backend communication or public web/API calls. Evidence: "https://otienomaurice.github.io",
Network request at line 5	Frontend-backend communication or public web/API calls. Evidence: "http://localhost:8080",
Network request at line 6	Frontend-backend communication or public web/API calls. Evidence: "http://localhost:8081",
Network request at line 7	Frontend-backend communication or public web/API calls. Evidence: "http://127.0.0.1:8080",
Network request at line 8	Frontend-backend communication or public web/API calls. Evidence: "http://127.0.0.1:8081"
Security or auth at line 11	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function json(data, status = 200, headers = {}) {
Security or auth at line 14	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: headers: {
Local storage at line 16	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: "Cache-Control": "no-store",
Security or auth at line 17	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ...headers
Security or auth at line 30	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function corsHeaders(request, env) {

Item	Explanation
Security or auth at line 31	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>const origin = request.headers.get("Origin") "";</code>
Security or auth at line 36	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>"Access-Control-Allow-Headers": "Content-Type",</code>
Cloudflare or AI at line 54	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>function extractOpenAiText(data = {}) {</code>
Security or auth at line 89	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>"Never invent portfolio projects, credentials, files, test results, or private repository access.",</code>
Cloudflare or AI at line 121	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>async function callOpenAi(body, env) {</code>
Cloudflare or AI at line 122	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>const model = env.OPENAI_MODEL "gpt-4.1-mini";</code>
Cloudflare or AI at line 124	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>model,</code>
Cloudflare or AI at line 135	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>max_output_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS 1000)</code>
Cloudflare or AI at line 138	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>const response = await fetch("https://api.openai.com/v1/responses", {</code>
Security or auth at line 140	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>headers: {</code>
Cloudflare or AI at line 141	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>"Authorization": `Bearer \${env.OPENAI_API_KEY}`,</code>
Cloudflare or AI at line 151	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>error: data?.error?.message "OpenAI request failed."</code>
Cloudflare or AI at line 156	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>answer: extractOpenAiText(data),</code>
Cloudflare or AI at line 157	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>model</code>
Cloudflare or AI at line 166	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>error: "Cloudflare Workers AI binding is not available for this Worker."</code>
Cloudflare or AI at line 170	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>const model = env.WORKERS_AI_MODEL "@cf/meta/llama-3.2-3b-instruct";</code>
Cloudflare or AI at line 171	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>const data = await env.AI.run(model, {</code>
Cloudflare or AI at line 173	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>max_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS 1000)</code>
Cloudflare or AI at line 178	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>model</code>
Cloudflare or AI at line 210	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>"The full AI model is not reachable from this Worker right now, so this is a fallback answer from the available context."</code>
Cloudflare or AI at line 214	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>"I can answer that in a general way, but the full AI model is not reachable from this Worker right now.",</code>
Network request at line 223	Frontend-backend communication or public web/API calls. Evidence: <code>async fetch(request, env) {</code>
Security or auth at line 224	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>const cors = corsHeaders(request, env);</code>
Security or auth at line 225	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>if (request.method === "OPTIONS") return new Response(null, { status: 204, headers: cors });</code>
Cloudflare or AI at line 228	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>if (request.method !== "POST" !url.pathname.endsWith("/api/portfolio-ai")) {</code>
Cloudflare or AI at line 243	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>let provider = env.OPENAI_API_KEY ? "openai" : "cloudflare-workers-ai";</code>
Cloudflare or AI at line 244	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>let result = env.OPENAI_API_KEY</code>
Cloudflare or AI at line 245	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>? await callOpenAi({ ...body, question }, env)</code>
Cloudflare or AI at line 247	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>if (!result.ok && env.OPENAI_API_KEY && env.AI) {</code>

Item	Explanation
Cloudflare or AI at line 248	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: provider = "cloudflare-workers-ai";
Cloudflare or AI at line 254	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: "worker-fallback",
Cloudflare or AI at line 262	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: result.model,
Cloudflare or AI at line 269	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: "worker-fallback",

Important constants and variables

Item	Explanation
defaultAllowedOrigins	Line 1: const defaultAllowedOrigins = [
origin	Line 31: const origin = request.headers.get("Origin") "";
allowOrigin	Line 32: const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
parts	Line 56: const parts = [];
messages	Line 96: const messages = [{ role: "system", content: buildInstructions() }];
model	Line 122: const model = env.OPENAI_MODEL "gpt-4.1-mini";
payload	Line 123: const payload = {
response	Line 138: const response = await fetch("https://api.openai.com/v1/responses", {
data	Line 146: const data = await response.json().catch(() => ({}));
model	Line 170: const model = env.WORKERS_AI_MODEL "@cf/meta/llama-3.2-3b-instruct";
data	Line 171: const data = await env.AI.run(model, {
question	Line 183: const question = String(body.question "").trim();
clean	Line 184: const clean = question.toLowerCase();
context	Line 185: const context = body.context {};
projects	Line 186: const projects = Array.isArray(context.projects) ? context.projects : [];
owner	Line 187: const owner = context.owner context.profile?.displayName "Maurice Otieno";
cors	Line 224: const cors = corsHeaders(request, env);
url	Line 227: const url = new URL(request.url);
body	Line 232: let body = {};
question	Line 239: const question = clamp(body.question, 1200).trim();
provider	Line 243: let provider = env.OPENAI_API_KEY ? "openai" : "cloudflare-workers-ai";
result	Line 244: let result = env.OPENAI_API_KEY

JSON/YAML-style keys detected

Item	Explanation
Content-Type	Line 15: "Content-Type": "application/json; charset=utf-8",
Cache-Control	Line 16: "Cache-Control": "no-store",
Access-Control-Allow-Origin	Line 34: "Access-Control-Allow-Origin": allowOrigin,
Access-Control-Allow-Methods	Line 35: "Access-Control-Allow-Methods": "POST, OPTIONS",
Access-Control-Allow-Headers	Line 36: "Access-Control-Allow-Headers": "Content-Type",
Access-Control-Max-Age	Line 37: "Access-Control-Max-Age": "86400",
Vary	Line 38: "Vary": "Origin"

Item	Explanation
Authorization	Line 141: "Authorization": `Bearer \${env.OPENAI_API_KEY}`,
Content-Type	Line 142: "Content-Type": "application/json"

Function-by-function detail

json (cloudflare/portfolio-ai-worker.js, lines 11-20)

json named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 11-20 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Response, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 12: return new Response(JSON.stringify(data, null, 2), {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
11: function json(data, status = 200, headers = {}) {
12:   return new Response(JSON.stringify(data, null, 2), {
13:     status,
14:     headers: {
15:       "Content-Type": "application/json; charset=utf-8",
16:       "Cache-Control": "no-store",
17:       ...headers
18:     }
19:   });
20: }
```

allowedOrigins (cloudflare/portfolio-ai-worker.js, lines 22-28)

allowedOrigins named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 22-28 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, trim, filter, and concat. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 23: return String(env.ALLOWED_ORIGINS || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
22: function allowedOrigins(env) {
23:   return String(env.ALLOWED_ORIGINS || "")
24:     .split(",")
25:     .map((origin) => origin.trim())
26:     .filter(Boolean)
27:     .concat(defaultAllowedOrigins);
28: }
```

corsHeaders (cloudflare/portfolio-ai-worker.js, lines 30-40)

corsHeaders named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 30-40 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, allowedOrigins, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 33: return {.

Important local values include origin at line 31; allowOrigin at line 32. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
30: function corsHeaders(request, env) {
31:   const origin = request.headers.get("Origin") || "";
32:   const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
33:   return {
34:     "Access-Control-Allow-Origin": allowOrigin,
35:     "Access-Control-Allow-Methods": "POST, OPTIONS",
36:     "Access-Control-Allow-Headers": "Content-Type",
37:     "Access-Control-Max-Age": "86400",
38:     "Vary": "Origin"
39:   };
40: }
```

clamp (cloudflare/portfolio-ai-worker.js, lines 42-44)

clamp named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 42-44 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 43: return String(value || "").slice(0, maxLength);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
42: function clamp(value, maxLength) {
43:   return String(value || "").slice(0, maxLength);
44: }
```

cleanConversation (cloudflare/portfolio-ai-worker.js, lines 46-52)

cleanConversation named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 46-52 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, slice, map, clamp, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 47: if (!Array.isArray(conversation)) return [];; line 48: return conversation.slice(-8).map((item) => ({

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
46: function cleanConversation(conversation = []) {
47:   if (!Array.isArray(conversation)) return [];
48:   return conversation.slice(-8).map((item) => ({
49:     role: item?.role === "assistant" ? "assistant" : "user",
50:     content: clamp(item?.content || item?.text || "", 1400)
51:   })).filter((item) => item.content);
52: }
```

extractOpenAiText (cloudflare/portfolio-ai-worker.js, lines 54-64)

extractOpenAiText named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 54-64 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 55: if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();; line 63: return parts.join("\n").trim();.

Important local values include parts at line 56. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
54: function extractOpenAiText(data = {}) {
55:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
56:   const parts = [];
57:   for (const item of data.output || []) {
58:     for (const content of item.content || []) {
59:       if (typeof content.text === "string") parts.push(content.text);
60:       if (typeof content.output_text === "string") parts.push(content.output_text);
61:     }
62:   }
63:   return parts.join("\n").trim();
64: }
```

extractWorkersAiText (cloudflare/portfolio-ai-worker.js, lines 66-79)

extractWorkersAiText named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 66-79 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, isArray, map, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 67: if (typeof data.response === "string" && data.response.trim()) return data.response.trim(); line 68: if (typeof data.result?.response === "string" && data.result.response.trim()) return data.result.response.trim(); line 69: if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim(); line 71: return data.output.map((item) => { There are 5 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
66: function extractWorkersAiText(data = {}) {
67:   if (typeof data.response === "string" && data.response.trim()) return data.response.trim();
68:   if (typeof data.result?.response === "string" && data.result.response.trim()) return
data.result.response.trim();
69:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
70:   if (Array.isArray(data.output)) {
71:     return data.output.map((item) => {
72:       if (typeof item === "string") return item;
73:       if (typeof item?.text === "string") return item.text;
74:       if (typeof item?.content === "string") return item.content;
75:       return "";
76:     }).filter(Boolean).join("\n").trim();
77:   }
78:   return "";
79: }
```

buildInstructions (cloudflare/portfolio-ai-worker.js, lines 81-93)

buildInstructions creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 81-93 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 82: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
81: function buildInstructions() {
82:   return [
83:     "You are the AI assistant for a public engineering portfolio website.",
84:     "Answer naturally like a helpful technical mentor and recruiter-facing portfolio guide.",
85:     "If the question is general, answer the concept directly before mentioning portfolio context.",
86:     "If the question is about the portfolio owner, projects, files, resume, tools, links, or sections,
answer from the supplied portfolio context.",
87:     "Use recent conversation to keep a conversational flow. Greetings should receive short greetings.",
88:     "Only include portfolio links when they are directly relevant to the visitor's question.",
89:     "Never invent portfolio projects, credentials, files, test results, or private repository access.",
```

```

90:     "If a requested detail is not in the context, say what is missing and give a useful next step.",
91:     "Keep answers concise, readable, and specific. Use bullets when they make the answer easier to scan."
92:   ].join("\n");
93: }

```

buildMessages (cloudflare/portfolio-ai-worker.js, lines 95-105)

This function converts an incoming chat request into model messages that include system instructions, prior conversation, user question, and portfolio context.

The quality of an AI answer depends heavily on the message structure. This function shapes the prompt before it reaches OpenAI or Workers AI.

It calls or references buildInstructions, cleanConversation, push, and buildUserPrompt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 104: return messages;.

Important local values include messages at line 96. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

95: function buildMessages(body = {}) {
96:   const messages = [{ role: "system", content: buildInstructions() }];
97:   for (const item of cleanConversation(body.conversation)) {
98:     messages.push({
99:       role: item.role === "assistant" ? "assistant" : "user",
100:      content: item.content
101:    });
102:   }
103:   messages.push({ role: "user", content: buildUserPrompt(body) });
104:   return messages;
105: }

```

buildUserPrompt (cloudflare/portfolio-ai-worker.js, lines 107-119)

buildUserPrompt creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 107-119 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clamp, stringify, cleanConversation, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 108: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

107: function buildUserPrompt(body = {}) {
108:   return [
109:     `Visitor question: ${clamp(body.question, 1200)}`,
110:     `Question intent: ${clamp(body.intent || "portfolio_specific", 80)}`,
111:     `Web/public lookup requested by browser: ${body.allowWebSearch === true ? "yes" : "no"}`,
112:     "",
113:     "Recent conversation JSON:",
114:     clamp(JSON.stringify(cleanConversation(body.conversation), null, 2), 6000),
115:     "",
116:     "Portfolio context JSON:",
117:     clamp(JSON.stringify(body.context || {}, null, 2), 20000)
118:   ].join("\n");
119: }

```

callOpenAi (cloudflare/portfolio-ai-worker.js, lines 121-145)

This function calls the OpenAI API from the Cloudflare Worker when an OpenAI key is configured, then extracts the answer from the provider response.

It keeps the API key server-side and gives the public website a stronger AI backend without exposing secrets in browser JavaScript.

It calls or references buildInstructions, buildUserPrompt, fetch, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include model at line 122; payload at line 123; response at line 138. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
121: async function callOpenAi(body, env) {
122:   const model = env.OPENAI_MODEL || "gpt-4.1-mini";
123:   const payload = {
124:     model,
125:     input: [
126:       {
127:         role: "developer",
128:         content: [{ type: "input_text", text: buildInstructions() }]
129:       },
130:       {
131:         role: "user",
132:         content: [{ type: "input_text", text: buildUserPrompt(body) }]
133:       }
134:     ],
135:     max_output_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
136:   };
137:
138:   const response = await fetch("https://api.openai.com/v1/responses", {
139:     method: "POST",
140:     headers: {
141:       "Authorization": `Bearer ${env.OPENAI_API_KEY}`,
142:       "Content-Type": "application/json"
143:     },
144:     body: JSON.stringify(payload)
145:   });
```

callWorkersAi (cloudflare/portfolio-ai-worker.js, lines 161-180)

This function calls Cloudflare Workers AI as a provider fallback when OpenAI is not configured.

It gives the website AI a deployable backend path even before or without an OpenAI Worker secret.

It calls or references run, buildMessages, and extractWorkersAiText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 163: return {}; line 175: return {}.

Important local values include model at line 170; data at line 171. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
161: async function callWorkersAi(body, env) {
162:   if (!env.AI || typeof env.AI.run !== "function") {
163:     return {
164:       ok: false,
165:       status: 503,
166:       error: "Cloudflare Workers AI binding is not available for this Worker."
167:     };
168:   }
169:
170:   const model = env.WORKERS_AI_MODEL || "@cf/meta/llama-3.2-3b-instruct";
171:   const data = await env.AI.run(model, {
172:     messages: buildMessages(body),
173:     max_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
174:   });
175:   return {
176:     ok: true,
177:     answer: extractWorkersAiText(data),
178:     model
179:   };
180: }
```

fallbackWorkerAnswer (cloudflare/portfolio-ai-worker.js, lines 182-220)

This function returns a simple local Worker answer when model calls are unavailable.

It prevents the public AI endpoint from returning a useless failure for basic questions or temporary provider outages.

It calls or references trim, toLowerCase, isArray, slice, map, filter, join, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 190: return `Hi, what can I do for you? I can help with \${owner}'s projects, files, resume links, or related electronics concepts.`; line 193: return []; line 204: return []; line 213: return [].

Important local values include question at line 183; clean at line 184; context at line 185; projects at line 186; owner at line 187. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
182: function fallbackWorkerAnswer(body = {}) {
183:   const question = String(body.question || "").trim();
```



```

184:   const clean = question.toLowerCase();
185:   const context = body.context || {};
186:   const projects = Array.isArray(context.projects) ? context.projects : [];
187:   const owner = context.owner || context.profile?.displayName || "Maurice Otieno";
188:
189:   if (/^(hi|hello|hey|yo)\b/.test(clean)) {
190:     return `Hi, what can I do for you? I can help with ${owner}'s projects, files, resume links, or
related electronics concepts.`;
191:   }
192:   if (/^bembbedded systems?\b/.test(clean)) {
193:     return [
194:       "Embedded systems is a field of engineering where software is built into dedicated hardware to
control a device, measure signals, communicate with peripherals, or respond to real-time events.",
195:       "",
196:       "Typical embedded work combines microcontrollers or processors, firmware, timing, interrupts,
communication buses, sensors, actuators, and hardware validation.",
197:       "",
198:       projects.length
199:       ? `In this portfolio, I would connect that explanation back to saved projects such as
${projects.slice(0, 3).map((project) => project.title || project.name).filter(Boolean).join(", ")} when those
projects are relevant.`
200:       : "When project context is available, I connect the explanation to the saved project evidence."
201:     ].join("\n");
202:   }
203:   if (projects.length &&
/^b(project|portfolio|maurice|file|github|resume|tool|design|test)\b/.test(clean)) {
204:     return [
205:       "I can use the portfolio context for ${owner}.",
206:       "",
207:       "Relevant saved project areas include:",
208:       ...projects.slice(0, 6).map((project) => ` - ${project.title || project.name || "Untitled
project"}`),
209:       "",
210:       "The full AI model is not reachable from this Worker right now, so this is a fallback answer from
the available context."
211:     ].join("\n");
212:   }
213:   return [
214:     "I can answer that in a general way, but the full AI model is not reachable from this Worker right
now.",
215:     "",
216:     "A useful answer should define the concept, explain the main parts, give a practical example, and
connect it to a project artifact when portfolio context is relevant.",
217:     "",
218:     `Question received: ${question}`
219:   ].join("\n");
220: }

```

Representative opening snippet

```

1: const defaultAllowedOrigins = [
2:   "https://mauriceotieno.com",
3:   "https://www.mauriceotieno.com",
4:   "https://otienomaurence.github.io",
5:   "http://localhost:8080",
6:   "http://localhost:8081",
7:   "http://127.0.0.1:8080",
8:   "http://127.0.0.1:8081"
9: ];
10:
11: function json(data, status = 200, headers = {}) {
12:   return new Response(JSON.stringify(data, null, 2), {
13:     status,
14:     headers: {
15:       "Content-Type": "application/json; charset=utf-8",
16:       "Cache-Control": "no-store",
17:       ...headers
18:     }
19:   });
20: }
21:
22: function allowedOrigins(env) {
23:   return String(env.ALLOWED_ORIGINS || "").
24:     .split(",")
25:     .map((origin) => origin.trim())
26:     .filter(Boolean)
27:     .concat(defaultAllowedOrigins);
28: }
29:
30: function corsHeaders(request, env) {
31:   const origin = request.headers.get("Origin") || "";
32:   const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
33:   return {
34:     "Access-Control-Allow-Origin": allowOrigin,
35:     "Access-Control-Allow-Methods": "POST, OPTIONS",
36:     "Access-Control-Allow-Headers": "Content-Type",
37:     "Access-Control-Max-Age": "86400",
38:     "Vary": "Origin"
39:   };
40: }

```

```

41:
42: function clamp(value, maxLength) {
43:   return String(value || "").slice(0, maxLength);
44: }
45:
46: function cleanConversation(conversation = []) {
47:   if (!Array.isArray(conversation)) return [];
48:   return conversation.slice(-8).map((item) => ({
49:     role: item?.role === "assistant" ? "assistant" : "user",
50:     content: clamp(item?.content || item?.text || "", 1400)
51:   })).filter((item) => item.content);
52: }
53:
54: function extractOpenAiText(data = {}) {
55:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
56:   const parts = [];
57:   for (const item of data.output || []) {
58:     for (const content of item.content || []) {
59:       if (typeof content.text === "string") parts.push(content.text);
60:       if (typeof content.output_text === "string") parts.push(content.output_text);
61:     }
62:   }
63:   return parts.join("\n").trim();
64: }
65:
66: function extractWorkersAiText(data = {}) {
67:   if (typeof data.response === "string" && data.response.trim()) return data.response.trim();
68:   if (typeof data.result?.response === "string" && data.result.response.trim()) return
data.result.response.trim();
69:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
70:   if (Array.isArray(data.output)) {
71:     return data.output.map((item) => {
72:       if (typeof item === "string") return item;

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.